

PacificA: Replication in Log-Based Distributed Storage Systems

林伟、毛洋 微软亚洲研究院
{weilin, maoyang}@microsoft.com

张林涛、周立东 微软硅谷研究院
{lintaoz, lidongz}@microsoft.com

抽象的

大规模分布式存储系统因存储和处理不断增加的数据量而受到欢迎。复制机制通常是在此类系统中实现高可用性和高吞吐量的关键。对共识等基本问题的研究为复制协议奠定了坚实的基础。然而，实际复制机制的架构设计和工程问题仍然是一门艺术。

本文描述了我们为常用的基于日志的存储系统设计和实现复制的经验。我们提倡简单、实用、强一致的通用复制框架。我们证明该框架足够灵活，可以适应我们探索的各种不同的设计选择。使用名为 PacificA 的原型系统，我们实施了三种不同的复制策略，全部使用相同的复制框架。该论文报告了详细的性能评估结果，特别是在故障、协调和恢复期间的系统行为。

1. 简介

人们越来越需要大规模存储系统来保存不断增加的数字化信息。为了提高成本效益，此类系统通常采用廉价的商品组件构建。因此，故障成为常态，复制等容错机制成为此类系统可靠性和可用性的关键。

众所周知且可证明正确的复制协议（例如 Paxos [18]）存在。然而，构建实用的分布式存储系统不仅仅是应用已知的复制协议。“理论”复制协议和实际复制机制之间存在显著差距。例如，复制协议的理论设计通常以过于简化的性能指标为指导，例如消息轮数，而实际的复制机制旨在优化端到端客户端感知的性能指标，例如系统吞吐量、请求处理延迟和故障后的恢复时间。理论设计侧重于复制协议的单个实例，而实际的复制机制必须优化具有多个实例的系统的整体性能。

共存的多个实例。

在本文中，我们描述了我们在局域网（LAN）集群环境中设计、实现和评估大规模基于日志的存储系统的复制机制的经验。这种基于日志的设计通常用于存储系统（例如，[12,13,15,24]），通过将随机写入转换为顺序写入、允许批处理以及支持事务语义来提高性能。许多最近提出的基于 LAN 集群的存储系统（例如 Petal [19]、Frangipani [28]、Boxwood [21] 和 Bigtable [7]）都使用日志。

基于日志的存储系统提供了丰富的结构，提供了有趣的设计选择供探索。例如，复制可以在不同的语义级别和不同类型的目标上完成（例如，逻辑状态、日志或磁盘上的物理数据）。

我们没有选择实现不同的复制机制来探索设计选择，而是选择设计和实现一个简单、实用且可证明正确的通用复制框架。然后，不同的选择转化为同一复制框架的不同实例。这显著减少了实施、调试和维护系统的开销，并有利于不同方案的公平比较。

复制框架的设计反映了我们坚信，可证明正确的复制协议是实际复制机制的重要基础。定义良好的系统模型阐明了协议工作的假设，并使我们能够理解和评估风险。相比之下，临时复制协议在一些未知的极端情况下通常会失败。

选择合适的复制协议以及选择正确的架构设计是良好实用复制机制的关键。我们相信简单性和模块化是构建实用系统的关键要素。我们的复制框架通过以下功能体现了这些原则：（i）将副本组配置管理与数据复制分离，使用 Paxos 来管理数据复制的配置和主/备份 [2, 22]，（ii）分散监控以检测故障并触发重新配置，监控流量遵循数据复制的通信模式。

阳离子，以及 (iii) 一个通用和抽象的模型，它阐明了正确性并允许不同的实际实例化。

在基于日志的分布式系统的背景下，复制框架还使我们能够探索与先前系统（例如 Boxwood 和 Bigtable）不同的新复制方案。特别是，我们的方案在简单性方面提供了吸引人的优势，因为不依赖单独的锁定服务，并且由于更好的数据共置而减少了网络流量。

为了评估所提出的复制方案，我们实现了 PacificA，这是一个基于分布式日志的系统原型，用于存储结构化和半结构化 Web 数据。我们进行了广泛的评估，以了解各种复制方案的性能和成本，无论是针对单个复制实例还是针对具有多个实例的整个系统。故障和恢复期间的系统行为在实践中很重要。我们对这些情况下的系统行为进行详细分析。

本文的结构如下。第 2 节描述了我们提出的复制框架。第 3 节介绍了该框架可应用于通用基于日志的存储系统的不同方法。太平洋地区复制方案的实施和评估是第 4 节的重点。第 5 节概述了相关工作。我们在第 6 节中得出结论。

2. 太平洋复制框架

在本文中，我们将注意力集中在基于局域网的集群环境中，其中系统通常由大量的 *servers* 组成。每台服务器都有一个或多个 CPU、本地内存以及一个或多个本地连接的磁盘。所有服务器都连接到高速局域网。我们假设有一个管理域，其中的服务器都是可信的。不考虑任何安全机制。

服务器可能会失败。我们假设故障停止故障。我们不假设服务器之间的消息延迟有界限，尽管在正常情况下它们往往很小。消息可以被删除或重新排序，但不能注入或修改。网络分区也可能发生。服务器上的时钟不一定是同步的，甚至是松散同步的，但时钟漂移被假定是有界的。

系统维护着一组（大）数据。每条数据都被复制到一组服务器上，称为 *replica group*。副本组中的每个服务器充当 *replica*。一台服务器可以作为多个组中的副本。数据复制协议遵循主/备份范例。副本组中的一个副本被指定为 *primary*，而其他副本被称为 *secondaries*。副本组的组成，指示谁是主副本，谁是从副本，被称为副本组的 *configuration*。由于副本故障或添加，副本组的配置发生变化；版本用于跟踪此类更改。

我们专注于实现 *strong con-* 的复制协议

sistency，因为它代表了最自然的一致性模型。通俗地说，强一致性确保复制系统的行为与其非复制系统相同，从而实现线性化等语义[14]。应对较弱的一致性模型通常会增加构建在复制层之上的应用程序的复杂性。在第 2.7 节中，我们讨论了强一致性的一种放松及其含义。

2.1 主备数据复制

我们采用主/备份范式进行数据复制。我们区分两种类型的客户端请求：不修改数据的 *queries* 和修改数据的 *updates*。在主/备份范例中，所有请求都发送到主服务器。主节点在本地处理所有查询，并让所有辅助节点参与处理更新。主/备份范式在实践中具有吸引力有几个原因。它很简单，非常类似于以主数据库作为唯一访问点的非复制情况。在许多工作负载中往往占主导地位的查询直接在单个服务器上处理。

Strong consistency can be achieved if all servers in a replica group process the same set of requests in the same order, assuming that updates are deterministic. 因此，主节点分配连续且单调递增的序列号来进行更新，并指示所有辅助节点按此顺序连续处理请求。为了清楚起见，我们将副本建模为维护请求的 *prepared list* 和列表的 *committed point*。该列表根据分配给请求的序列号进行排序，并且是连续的（即序列号为 sn 的请求仅在插入 $sn - 1$ 之后插入。）到提交点的准备列表的前缀称为 *committed list*。我们在 2.6 节中描述了抽象模型的实际实现。

主节点上的应用程序状态是按初始状态上的序列号升序应用已提交列表中的所有请求的结果。只要系统没有遇到无法容忍的故障（例如，当前配置中所有副本的永久故障），已提交列表上的请求就保证不会丢失。准备列表的未提交足够用于确保已提交列表上的请求在重新配置期间得以保留。我们使用 $committed_p$ 表示服务器 p 的已提交列表的更新集，使用 $prepared_p$ 表示 p 的准备列表。

正常情况查询和更新协议。在正常情况下（即无需重新配置），数据复制协议很简单。当主节点收到查询时，它会根据当前提交列表表示的状态处理查询并立即返回响应。

对于更新，主 p 会为请求分配下一个可用序列号。然后，它发送请求及其配置版本和 *prepare* 中的序列号

向所有副本发送消息。收到 *prepare* 消息后，副本 *r* 会将请求按序列号顺序插入到其准备好的列表中。该请求在副本上被视为 *prepared*。然后，*r* 在 *prepared* 消息中向主节点发送确认。当主节点收到所有副本的确认时，请求为 *committed*。此时，主节点将其提交点向前移动到最高点，以便提交到此点的所有请求。然后 *p* 向客户端发送确认，表明成功完成。对于每条 *prepare* 消息，主节点进一步在提交点搭载序列号，以通知辅助节点有关已提交的所有请求。然后，辅助节点可以相应地向前移动其承诺点。

因为主节点仅在所有副本都将请求插入其准备列表后才将请求添加到其提交列表中（当向前移动提交点时），因此主节点上的提交列表始终是任何副本上准备列表的前缀。此外，由于辅助设备仅在主设备完成此操作后才将请求添加到其提交列表中，因此辅助设备上的提交列表保证是主设备上的提交列表的前缀。因此，这个简单的数据复制协议支持以下提交不变式。

提交不变：令 *p* 为主节点，*q* 为当前配置中的任何副本， $\text{committed}_q \subseteq \text{committed}_p \subseteq \text{prepared}_q$ 成立。

仅当副本组的配置没有变化时，基本的主/备协议才起作用。我们的复制框架将配置管理与数据管理分开。我们在接下来的小节中描述配置管理及其与数据复制的交互。

2.2 配置管理

在我们的设计中，*global configuration manager* 负责维护系统中所有副本组的配置。对于每个副本组，配置管理器维护当前配置及其版本。

当服务器 *suspects* (通过故障检测) 发现副本有故障时，可以启动重新配置；重新配置将从配置中删除有故障的副本。服务器还可以建议向配置中添加新的副本；例如，恢复理想的裁员水平。在这两种情况下，服务器都会将建议的新配置与其当前配置版本一起发送到配置管理器。当且仅当版本与配置管理器上当前配置的版本匹配时，该请求才会被接受。在这种情况下，建议的新配置将随下一个更高版本一起安装。否则，请求将被拒绝。

在网络分区导致主节点与辅助节点断开连接的情况下，可能会出现冲突的重新配置请求：主节点可能希望删除某些辅助节点，而某些辅助节点则尝试删除

主要的。因为所有这些请求都基于当前配置，所以配置管理器接受的第一个请求获胜。所有其他冲突请求都会被拒绝，因为配置管理器已经升级到具有更高版本的新配置。

任何确保以下主不变量的故障检测机制都可以用来触发当前副本的删除。在 2.3 节中，我们详细描述了这种故障检测机制的一种实现。

主不变量：在任何时候，只有当配置管理器将 *p* 作为其维护的当前配置中的主服务器时，服务器 *p* 才会认为自己是主服务器。因此，在任何时候，最多存在一台服务器将自己视为副本组的主服务器。

2.3 租赁和故障检测

即使配置管理器维持当前配置的真实性，主不变量也不一定成立。这是因为不同服务器上的本地配置视图不一定同步。特别是，我们必须避免旧主节点和新主节点同时处理查询的情况——旧主节点可能不知道重新配置已创建新主节点并将其从配置中删除。由于新的主数据库可能已经处理了新的更新，因此旧的主数据库可能正在处理过时状态的查询，从而违反了强一致性。

我们的解决方案是使用 *leases* [11]。主节点通过定期向辅助节点发送信标并等待确认来从每个辅助节点获取租约。如果自上次确认信标的发送时间以来已经过去了指定的 *lease period*，则主节点认为其租约已过期。当辅助节点的任何租约到期时，主节点不再认为自己是主节点并停止处理查询或更新。在这种情况下，主节点将联系配置管理器以从当前配置中删除辅助节点。

只要发送者在当前配置中仍然是主发送者，辅助发送者就会确认信标。如果自上次从主节点接收到信标后 *grace period* 已经过去，则辅助节点会认为对主节点的租约已过期，并将联系配置管理器以删除当前主节点并成为新的主节点。

假设时钟漂移为零，只要宽限期等于或长于租用期，租用就可以保证在主服务器上先于辅助服务器到期。当且仅当其对旧主节点的租约到期时，辅助节点才会提出配置更改以尝试承担主节点的角色。因此，保证在新的 Primary 建立之前，旧的 Primary 已经退出；因此初级不变量成立。

我们使用租赁机制作为故障检测机制。其他系统中也使用了类似的故障检测机制，例如 GFS [10]、Boxwood 和 Big table/Chubby

[5]。这些系统的一个主要区别是租赁是从集中式实体获得的。在我们的例子中，用于故障检测的监控流量始终发生在两个已经需要相互通信以进行数据处理的服务器之间：主服务器在处理更新时与辅助服务器通信；信标和确认也在主节点和辅助节点之间进行。这样，故障检测就可以准确捕获复制所需的通信通道的状态。此外，当通信通道繁忙时，数据处理消息本身可以充当信标和确认。实际的信标和确认仅在通信通道空闲时发送，从而最大限度地减少故障检测的开销。此外，这种分散的实施消除了对集中实体的负载和依赖。负载可能很大，因为信标和确认会在中央实体和系统中的每个服务器之间定期交换；该间隔必须相当小，以便能够快速检测故障。在集中式解决方案中，集中式实体的不可用（例如，由于网络分区）可能会导致整个系统不可用，因为所有主节点在丢失来自中央实体的租约时都必须辞职。

2.4 重新配置、协调和恢复

复制协议的复杂性在于它如何处理重新配置。我们将重新配置分为三种类型：删除辅助设备、删除主设备和添加辅助设备。

去除次级。当主节点怀疑辅助节点的子集出现故障时，就会触发删除一个或多个辅助节点。主节点提议一个新的配置，排除那些可疑的辅助节点，并在配置管理器批准该提案后继续排除这些辅助节点。

主要的变化。当辅助设备使用前面描述的租用机制怀疑主设备发生故障时，将触发主设备的删除。次要节点提出一个新的配置，其本身就是新的主要节点，旧的主要节点被排除在外。获得批准后，辅助节点 p 会将自己视为新的主节点，并且仅在完成 *reconciliation* 流程后才开始处理新请求。在协调期间， p 会针对其准备列表中未提交的请求发送 *prepare* 消息，并让它们在新配置上提交。令 sn 为协调后与 p 提交点对应的序列号，然后 p 指示所有副本截断其准备好的列表，并仅保留序列号 sn 之前的所有请求。即使在协调期间，如果新配置中的任何副本失败，也可以触发另一个重新配置。

由于辅助节点上的准备列表始终包含所有已提交的请求，因此通过提交准备列表中的所有请求，协调可确保以下 Recon-

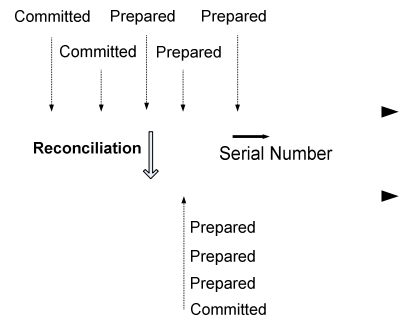


图 1：调节：示例。A 是旧配置中的主要。在新配置中，旧的辅助节点 B 被提升为新的主节点，同时从配置中删除了 A。第一行根据列表上的最高序列号显示副本的准备列表和已提交列表的状态。第二行显示协调后对应的状态。

图形不变量。

重新配置不变式：如果新的主节点 p 在时间 t 完成协调，则副本组中的任何副本在 t 之前的任何时间维护的任何已提交列表都保证在时间 t 时为 committed_p 的前缀。提交不变性在新配置中保持不变。

重新配置不变性扩展了跨配置的提交不变性。它还确保分配给任何已提交请求的序列号将被保留，并且即使发生主要更改也不会重新分配。相反，准备好的列表的未提交部分中的请求不一定被保留。例如，当分配该序列号的主节点失败时，只要该序列号尚未分配给已提交的请求，新的主节点就可以将相同的序列号分配给不同的请求。辅助节点始终选择来自具有最高配置版本的主节点的分配。

图 1 说明了当主项发生变化时，准备好的列表和提交的点是如何变化的。最初，A 是主要的，B、C 和 D 作为辅助。请注意， committed_B 是 committed_A 的前缀，而 committed_A 是任何副本上准备好的列表的前缀。考虑使用 B 替换失败的 A 作为主配置的重新配置。B 完成协调后，新的 committed_B 与旧的 prepared_B 相同，其中包含 committed_A 。 prepared_C 已更新以保留提交不变性。 prepared_D 上的额外请求已删除（或撤消）。¹

添加新的次级。可以将新的副本添加到副本组中，以恢复组中副本发生故障后的冗余级别。当新服务器添加到配置中时，必须保留提交不变性。

¹In some cases, the truncation of prepared_D can be done lazily when D receives from the new primary B new requests with the same serial numbers. Those requests will replace the existing ones.

这要求新副本在加入副本组之前具有适当的准备列表。

²新副本获取正确状态的过程称为*recovery*。

一种简单的方法是让主节点延迟处理任何新更新，直到新副本从任何现有副本复制准备好的列表。这在实践中往往会造成破坏。或者，新副本作为 *candidate* 辅助副本加入。主节点继续处理更新并向候选辅助节点发送 *prepare* 消息。如果主节点未能从新副本获得确认，则主节点将终止该副本的候选资格。

除了在来自主节点的 *prepare* 消息中记录新的更新之外，候选辅助节点 *c* 还会从任何现有副本中获取它尚未拥有的准备列表部分。当 *c* 最终赶上时，它会要求主节点将 *c* 添加到配置中。只要 *c* 的候选资格尚未终止，主节点就会联系配置管理器以将 *c* 作为辅助节点添加到配置中。获得批准后，主节点将 *c* 视为辅助节点。

在恢复期间将状态转移到新副本的成本可能很高。对于从未加入过副本组的副本来说，完整的状态转移是不可避免的。在某些情况下，由于错误怀疑、网络分区或其他暂时性故障，副本会从副本组中删除。当此类过时的副本重新加入组时，理想的情况是让新副本执行 *catch-up* 恢复 [26]，这仅获取副本离开组时已处理的丢失更新。由于重新配置不变，旧副本的提交列表保证是当前提交列表和当前准备列表的前缀。但是，如果旧副本离开组后发生任何主更改，则不能保证旧副本上的准备列表是当前准备列表的前缀。因此，旧副本只能保留其准备好的列表，直至追赶恢复中的提交点，并且必须获取其余部分。

2.5 正确性

如果在任何时候配置管理器定义的当前配置中至少存在一个无故障副本，那么我们在本节中描述的协议就实现了强一致性。我们将复制协议的完整描述及其正确性的正式证明留给单独的技术报告。在这里，我们提供了线性化、耐久性和进度正确性的非正式论证。

线性化：系统执行相当于所有已处理请求的线性执行。

持久性：如果客户端收到系统对更新的确认，则该更新已包含在

²In practice, the joining server copies the current committed state, as well as the part of the prepared list that has not been committed.

等效线性化执行。

进展：假设无故障服务器之间的通信链路是可靠的，配置管理器最终响应重新配置请求，并且最终系统中没有故障和重新配置，系统将返回对每个客户端请求的响应。

对于线性化，请观察主数据库上的提交列表定义了所有更新的线性化执行。Primary Invariant 定义了一个副本组的一系列 *primary*，每个 *primary* 都在不重叠的时间段内起作用。即使主节点发生变化，由于重新配置不变，任何先前主节点上的已提交列表也保证是新主节点上已提交列表的前缀。要将查询添加到线性化执行中，请观察主进程处理由已提交列表表示的状态上的每个查询；该查询自然会添加到该已提交列表中的所有已提交更新之后以及所有先前完成的查询之后。同样，由于主不变，在任何时候，单个主都在处理所有查询。查询的添加保留了线性化执行。很容易看出，线性化执行满足线性化所需的实时排序。

对于持久性，如果客户端收到更新确认，则发送确认的主设备必须已将请求插入到其已提交列表中，因此处于等效的线性化执行中。

对于 Progress，假设配置管理器最终返回每个重新配置请求的响应（Paxos 在合理的时序假设下这样做），副本组最终会重新配置为仅包含无故障副本的配置。此时，所有请求都将根据数据复制协议进行处理和确认。

2.6 实现

实际上，副本并不维护抽象模型中描述的准备列表和提交点。它们的最佳实现方式取决于所复制的数据（或应用程序状态）的语义。基于复制协议，实现必须有效支持三种类型的操作。

- 查询处理：处理与提交列表对应的状态的查询。
- 协调期间的状态转移：使副本上准备好的列表与新的主副本相同。由于状态转移，辅助节点上准备好的列表中的一些未提交的条目可能会被删除。
- 恢复期间的状态传输：将准备好的列表的所需部分从副本传输到加入服务器。由于状态转移，加入服务器上准备好的列表中的一些未提交的条目可能会被删除。

允许高效处理查询的自然方法是将已提交的请求按序列号顺序应用到应用程序状态。尚未提交的准备好的请求可能需要单独维护：这些请求不会直接应用于应用程序状态，因为它们可能需要在状态传输期间撤消：一般来说，撤消操作很棘手，可能需要单独的撤消日志。

为了协调，新的主节点只需发送所有准备好的和未提交的请求；它们很容易识别，因为它们与应用程序状态分开维护。为了加速追赶恢复，副本还可以在内存或磁盘上维护最近提交的更新。或者，他们可以通过脏区域日志记录来标记由最近提交的更新修改的状态段。

在此设计中，要提交更新，辅助节点必须首先在准备好的列表中记录更新，然后在得知请求已提交时将更新应用到应用程序状态。这通常涉及每个提交的请求两次磁盘写入。

一种优化是批量更新应用程序状态，方法是首先将更新应用于内存中的状态映像，然后定期将新映像写入磁盘。单独的准备列表不仅保留尚未提交的准备请求，还保留尚未反映在磁盘应用程序状态上的已提交请求。当内存状态因断电而丢失时，需要后者来恢复已提交的更新。

设计仅附加应用程序状态。在系统维护仅附加应用程序状态的情况下，稍后的更新附加到旧状态，而不是覆盖它，准备好的列表可以直接在仅附加应用程序状态上维护。这是因为撤消对仅附加状态的更新非常简单。

在此设计中，当从主服务器接收到请求时，辅助服务器按串行顺序将请求直接应用于应用程序状态，从而有效地使用应用程序状态来维护其准备好的列表。在提交请求之前，主节点仍然等待所有辅助节点的确认，从而有效地使用应用程序状态作为其提交列表。当从主节点获取有关提交点的捎带信息时，辅助节点会惰性地维护提交点。

为了协调，当新的主节点 p 启动时，每个辅助节点 s 确保提交点之后的状态部分与 p 上的状态部分相同。为了恢复，返回的副本只需从当前主副本的提交点开始复制所有内容。

2.7 讨论

在本小节中，我们讨论框架的一些设计细节并探索替代设计选择。

配置管理的可用性和性能

老化。拥有集中式配置管理器可以简化系统管理，因为它是维护系统中所有副本组配置的当前事实的中心点。更重要的是，这种分离还简化了数据复制协议，并允许在大小为 n 的副本组中容忍 $n - 1$ 故障。

为了实现高可用性，配置管理器本身是使用复制状态机方法和标准 Paxos 协议来实现的。该服务通常部署在少量（例如 5 或 7 台）固定服务器上，并且可以容忍不到一半的服务器不可用。与集中式锁定服务不同，配置管理器的不可用不会影响其他服务器的正常操作。

配置管理器不太可能成为性能瓶颈。配置管理器不参与正常情况下的请求处理，只在副本组重新配置时参与。通常，Paxos 在单轮通信中处理命令，每个成员服务器将更改提交到本地磁盘。可以采取进一步的措施来减少配置管理器的负载。通常，对配置管理器的每个请求（即使是读取）都需要执行共识协议。然而，在我们的例子中，配置管理器的任何成员都可以根据其本地存储的配置版本自由拒绝过时的配置更改请求，因为最新的配置至少具有相同的版本或更高版本。

临时副本组故障期间的持久性。在前面的部分中，我们展示了复制协议能够容忍大小为 n 的副本组中的 $n - 1$ 次故障。在实际应用中，有时会出现断电等情况，导致副本组中的所有服务器都出现故障。在这种情况下，副本组将无法取得进展，直到组中的某些副本重新上线。如果最新配置中的所有副本都永久失败，则会发生数据丢失。如果最新配置中的至少一个副本恢复在线，只要配置管理器也从电源故障中恢复，我们的复制协议就允许系统在不丢失数据的情况下恢复。对于使用 Paxos 协议实现的配置管理器，如果配置管理器的大多数服务器恢复并且所有 Paxos 状态都保存在持久存储上，则管理器可以恢复。

为了实现整个副本组短暂故障的持久性，所有副本都在持久存储上维护其准备列表和提交点。当副本从故障中恢复时，它会联系配置管理器以查看它是否仍然是当前配置中的副本。如果是这样，它将承担其当前角色；如果它是主要节点，它将尝试从辅助节点重新获得租约；如果它是辅助设备，它将响应来自当前主设备的信标。服务器在检测到故障时启动重新配置。

主/备份与 Paxos。使用主/备份范例的替代方案是采用 Paxos 范例。

在 Paxos 中，只有在法定数量（通常是大多数）副本上准备好请求后，才会提交请求。即使对于查询请求也是如此，尽管我们协议中的相同租赁机制可用于允许对持有仲裁租约的单个副本进行查询。

在主/备份范例中，只有在所有副本上准备好更新请求后才会提交更新请求，而不是像 Paxos 中那样在大多数副本上准备好。两个协议中法定人数的不同选择有几个含义。

首先，Paxos 对与少数副本相关的瞬态性能问题不敏感：它能够在最快的多数副本上提交请求；其他人可以在后台赶上。同样，少数副本的故障对性能的影响也很小。

相反，在我们的主/备份协议中，任何副本的缓慢都会导致协议变慢。任何副本的故障都会导致进度停滞，直到重新配置将该副本从副本组中删除为止。必须适当设置租约的超时值：较高的值会在副本故障期间转化为较长的中断时间，而较低的值会导致可能的错误怀疑，从而从副本组中删除无故障的慢速副本。错误的怀疑会降低系统的弹性；我们的协议有助于快速追赶恢复，使副本能够快速重新加入，从而减少漏洞窗口。

其次，当大多数副本不可用时，Paxos 无法处理任何请求。然而，在我们的协议中，副本组可以在配置管理器的帮助下重新配置，并且只要至少有一个副本可用，就可以继续运行。

第三，在单独的配置管理器的帮助下，我们的协议中的重新配置很简单。使用 Paxos，副本组可以通过将重新配置视为当前配置中的共识决策来重新配置自身。新副本必须联系当前配置中的多数副本才能正确传输状态。

在实践中哪种选择提供更好的权衡是有争议的。我们选择主/备份范例主要是因为它的简单性。即使采用 Paxos 范式，我们报告的大多数经验和结果仍然有效。

弱化强一致性。强一致性本质上有两个要求。首先，所有副本必须以相同的顺序执行同一组更新。其次，查询返回最新状态。放松强一致性可能会有好处。放宽第一个要求会导致副本之间的状态转移，这通常需要系统检测并恢复到一致的状态，并且还需要客户端处理不一致的情况。此类系统的示例包括 Coda [16]、Sprite [3]、Bayou [27]、CONIT [30]、PRACTI [4] 和 GFS。

相反，我们讨论第二个的放松；也就是说，我们允许查询返回有点过时的状态。这有两个有趣的含义。首先，复制协议在不依赖租赁机制的情况下实现了这种弱化语义：请注意，主不变量是

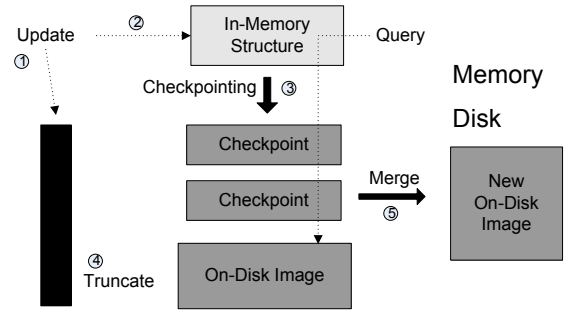


图 2：基于日志的存储系统架构。

依赖于租赁机制中的时间假设的唯一不变量。如果没有这个不变量，旧的主节点可能会错误地认为它仍然是主节点，并处理对过时状态的查询。然而，两个并发主节点的存在不会导致不同副本上的状态冲突。这是因为我们的重新配置协议规定新的主节点是先前配置中的副本。鉴于只有在配置中的所有副本都同意后会提交更新，只要新的主节点不再充当旧配置中的辅助节点，在新的主节点获得配置管理器批准后就无法再提交更新。因此，即使在主要变化期间也不会发生状态分歧。

第二个含义是辅助节点也可以处理查询。由于提交不变性和重新配置不变性，辅助节点上的提交列表表示有效但过时的状态。

3. 基于日志的分布式存储系统的复制

我们通过基于日志的分布式存储系统探索所提出的复制框架的实用方面。

图 2 显示了（非复制）基于日志的存储系统的一般架构。系统维护应用程序数据的磁盘映像以及捕获磁盘映像更新的内存数据结构。为了持久性，系统还为仅在内存中维护的更新保留一个 *application log*。

当系统收到更新时，它将请求写入日志以实现持久性（步骤 1），并将请求应用于内存中的数据结构（步骤 2）。当服务器重新启动时，日志将被重播以重建内存中的状态。定期（在内存数据结构用完内存之前），为内存数据结构在磁盘上创建检查点（步骤 3）。创建检查点后，可以从日志中截断磁盘上检查点中反映的所有更新日志条目（步骤 4）。系统可以选择存储一个或多个检查点。定期地将检查点中反映的更新与磁盘映像合并以创建新的

磁盘映像（步骤 5）。系统可以选择不带检查点的合并：内存中数据结构中反映的更新直接与磁盘上的映像合并。我们选择更通用的形式，因为它很好地涵盖了特殊情况，因为它准确地反映了一些实际系统（例如 Bigtable）的工作情况，并且因为检查点的维护可以在某些情况下加快协调速度。使用内存数据结构、磁盘检查点和磁盘映像来提供查询。值得指出的是，这种基于日志的设计消除了随机磁盘写入，并将其转化为更高效的顺序磁盘写入。

3.1 逻辑复制

在第一种复制策略中，复制只是应用于整个系统状态：每个副本支持与原始非复制系统相同的接口，并且能够处理相同类型的更新和查询。它是 *logical*，因为副本上的状态在逻辑上是相同的。复制品可能会保持不同的物理状态；副本可以单方面决定何时检查点以及何时合并。

对于逻辑复制，由于应用程序状态不是仅追加的，因此我们选择将准备好的列表与应用程序状态分开维护。重要的是，我们可以通过维护一个既充当应用程序日志又充当准备请求存储的日志来消除向准备列表写入更新的成本。为此，我们增加了一个包含客户端请求的应用程序日志条目，其中包含三个字段：配置版本、序列号和最后提交的序列号。日志条目包含具有相同序列号和较低配置版本的任何其他条目；当准备好的请求在主要更改期间被撤销时，就会发生这种情况。因此，应用程序日志由具有已提交序列号的所有未包含的日志条目组成，而未提交的日志条目属于准备好的列表。

在第一阶段，当接收到包含请求、配置版本、序列号和最后提交的序列号的 *prepare* 消息时，副本会将消息附加到日志中。在第二阶段，当提交请求时，它会被应用到内存中的数据结构，但应用程序不再需要写入日志：请求已经在日志中了。³

应用程序在检查点后截断其日志。但只有与已提交和检查点更新相对应的日志条目才会被截断。我们的复制协议只需要准备列表的已提交后缀。所有这些条目在截断期间确实被保留。

每个检查点反映一系列序列号的更新；该范围与检查点相关联。我们也

³On a primary, although a committed request is immediately applied to the in-memory data structure, the last committed serial number is updated lazily on the log. This does not affect correctness because our protocol ensures that any request in the prepared lists of all replicas will not be lost.

将磁盘映像与其反映的最后一个序列号关联起来。这些关联的序列号可以帮助确定追赶恢复期间需要哪一个状态。

逻辑复制的一种变体。处理内存中数据结构的更新、生成检查点以及合并所有消耗的资源（例如 CPU 周期和内存），并且可能会很昂贵。在逻辑复制中，这些操作既在主数据库上完成，又在辅助数据库上冗余地完成。

最好只让主服务器执行这些操作。然后可以将这些操作的结果复制到所有副本上。因此，我们引入了逻辑复制方案的变体，称为 *logical-V*。在逻辑-V 中，只有主节点维护内存中状态。当从主节点接收到更新时，辅助节点只需将请求附加到日志中，就像在逻辑复制中一样，而不是将请求应用于内存中状态。辅助节点会在生成检查点时从主节点获取检查点。仅当与条目对应的更新被丢弃或提交并反映在本地检查点中时，辅助日志条目才能被丢弃。

Logical-V 代表了逻辑复制的一个有趣的权衡。在逻辑-V 中，辅助节点消耗的内存要少得多（因为它们不再维护内存中的结构）和 CPU（因为它们不再生成涉及压缩的检查点）。但是，逻辑-V 确实会产生更高的网络负载，因为检查点现在在副本之间传输。更重要的是，逻辑 V 中的主节点故障转移更加耗时：要成为新的主节点，辅助节点必须重播日志以重建内存状态并生成所需的检查点。

3.2 分层复制

基于日志的系统中的持久状态由检查点、磁盘映像和日志组成。它们都可以作为文件来实现。基于日志的存储系统可以被认为由两层组成，其中下层提供持久文件存储，上层将应用程序逻辑转换为对文件的操作。

分层的观点导致了另一种设计，即分别将复制应用于两层。在较低层，一个简单的仅附加文件抽象足以支持附加新日志条目和创建新检查点等操作。因此，我们可以使用 2.6 节中概述的仅附加状态设计来进行文件复制。

具有较低层的文件复制也很有吸引力，因为抽象是可重用的并且简化了上层的逻辑。RAID 是此类设计的典型示例。最近的分布式存储系统，例如 Petal 上的 Frangipani、RLDev 上的 Boxwood 和 GFS 上的 Bigtable，都使用分层方法，并且主要复制逻辑位于较低层。

在上层，对于每一条数据，都有一些 *owner* 服务器负责接受客户端对该数据的请求，

维护内存中的结构，并写入（附加）到较低层的文件以实现持久性和可靠性。例如，要写入日志条目，上层只需请求下层将该条目附加到日志文件中即可。上层并不关心日志在哪里维护、是否复制、如何复制。下层不知道也不关心正在操作的文件的语义，无论该文件是日志还是检查点。

下层复制并不能完全消除上层容错的需要：当所有者服务器发生故障时，新的所有者服务器必须接管其责任。在以前的系统中，使用单独的分布式锁服务来确保上层为每条数据选择唯一的所有者服务器。我们的一个重要观察是，无需发明和部署新机制，相同的复制逻辑可以在上层重用，以选择唯一所有者来接受和处理客户端请求：选择唯一所有者的问题与实现主不变性的复制框架的配置管理部分相同。

一个自然的设计是选择管理较低层数据的副本组的主服务器作为所有者服务器。有两个主要优点。首先，我们从下层的复制协议中获得上层的“免费”故障转移机制，无需分布式锁服务，也不会产生任何额外的开销。当所有者服务器发生故障时，将在较低层选举出新的主服务器作为新的所有者服务器。其次，这种安排确保数据的副本始终驻留在所有者服务器上，从而减少网络流量。

3.3 日志合并

系统通常会维护多个副本组。正如 GFS 和链复制 [29] 所提倡的，服务器参与一组不同的副本组，以便在服务器发生故障时分散恢复负载。鉴于逻辑上存在与每个副本组关联的日志，每个服务器必须维护多个此类日志。使用日志记录的好处可能会减少，因为写入多个日志会导致磁盘头定期寻道。

一种解决方案是将多个日志合并到一个物理日志中，就像 Bigtable 中所做的那样。复制方式的选择会影响日志的合并方式。对于逻辑和逻辑-V，日志条目显式发送到辅助节点。然后，每个服务器可以将日志条目放入单个本地物理日志中。生成的本地物理日志包含该服务器所属的所有副本组的日志条目，无论该服务器是该组的主服务器还是辅助服务器。在该方案中，系统中的每台服务器都只有一个本地物理日志。

对于分层复制，服务器将其拥有的所有数据的日志条目作为主数据写入较低层的单个复制日志。这有效地合并了服务器作为主服务器的所有副本组的日志，但不合并服务器作为辅助服务器的副本组的日志。

系统中有 n 个服务器，每个服务器都是某个副本组的主服务器，在分层方案中将有 n 个复制日志。如果每个日志被复制 k 次，则系统中将有 kn 个单独的本地日志。这与逻辑和逻辑-V 情况下的 n 本地日志形成对比。在分层方案中，不能保证故障转移期间新主数据库所需的日志条目是本地的。这是因为不同副本组的日志条目被合并到一个日志中。这些不同的副本组通常在不同的服务器上有其辅助副本，以实现并行恢复。因此，日志的辅助副本不能与所有这些辅助副本位于同一位置。相反，在逻辑-V 情况下，在故障转移期间，保证所需的所有日志条目都是本地的，但每个服务器必须解析合并的日志文件以查找与新主副本相关的必要日志条目。对于逻辑复制，当副本从断电重新启动时，副本会重播其本地日志以重建内存中状态，但新的主节点在故障转移期间不需要重播其本地日志，因为它会维护内存中状态。

4. 评估

为了评估复制框架并评估它在实际的基于日志的分布式存储系统中的工作情况，我们实现了 PacificA，一个基于日志的存储系统原型。此处仅描述与复制相关的功能。PacificA 实现了一个分布式存储系统，用于存储大量的 *records*。每条记录都有一个唯一的 *key*；为所有键定义了总顺序。记录被分割成 *slices*，每个切片由连续键范围内的所有记录组成。PacificA 为客户端提供了一个接口，用于扫描给定键范围内的记录、查找特定键的记录、添加具有特定键的新记录或修改/删除现有记录。

PacificA 的设计遵循图 2 所示的架构，并采用 Bigtable 中的各种功能。在切片的磁盘映像中，切片中的记录按照其键的排序顺序连续存储。维护索引结构和布隆过滤器以实现快速查找。所有修改都会记录并反映在内存数据结构中。内存中的数据结构也是根据键进行排序的。当内存中的数据结构增长到一定大小时，数据结构中的信息被压缩并以相同的排序顺序写入持久存储。当将检查点与磁盘上的映像合并时，排序顺序允许进行简单的合并排序。为了减少写入磁盘的数据量，使用 ZLib（压缩级别 1）来压缩检查点和磁盘映像中的数据。

我们全面实现了三种不同的复制策略：逻辑复制、逻辑V复制和分层复制。由于使用相同的复制框架，代码库的很大一部分对于不同的策略是通用的。

评估是在 28 台机器组成的集群上完成的

通过 48 端口 1GB 以太网交换机连接。其中 16 台机器用作 PacificA 服务器；其他用作客户端计算机。这 16 台服务器均配备 2.0 GHz Intel Xeon 双核 CPU、4096 MB 内存、1 Gb 以太网卡和 232 GB SATA 硬盘。在我们的实验中，我们让配置管理器在单独的机器上以非复制方式运行。我们将租用期设置为 10 秒，宽限期设置为 15 秒。

由于查询仅由主数据库提供服务，因此复制方案的评估主要集中在更新性能上。为了生成具有更新的工作负载，客户端随机生成一个 26 字节密钥和 32 KB 的记录，压缩率约为 65%。大小和压缩比模仿了我们拥有的一组 100 GB 爬网网页。我们已经对以 URL 作为键插入真实网页数据的工作负载进行了实验，并发现了类似的结果。合成工作负载允许客户端有效地生成负载（无需从磁盘加载网页并首先解压缩数据）以使服务器饱和。对于报告更新吞吐量的所有实验，我们始终有足够的客户端和足够的线程发送随机生成的更新以使服务器饱和。组提交在所有方案中均已实现，并且默认开启。副本组的默认副本数为 3。

4.1 正常情况下的性能

在第一个实验中，客户端发送存储在由三台服务器组成的副本组上的单个片上的更新。本实验中不执行任何合并操作，并且组提交被禁用。表 1 显示了所有三种复制方案的客户端观察到的吞吐量，以及副本上复制的内部成本（在 CPU 利用率、磁盘吞吐量和网络利用率方面）。相比之下，非复制设置的客户端吞吐量为 23.7 MB/s，由于没有复制成本，比逻辑 V 的客户端吞吐量高出约 10%。我们显示了主服务器和辅助服务器上的资源消耗。报告的数字是 10 分钟内的平均值。

两个辅助节点上的资源消耗实际上是不同的。这主要是因为我们的实现使用主节点的链式结构来向辅助节点发送请求，就像 GFS 中所做的那样：主节点将数据发送到第一个辅助节点，然后该节点将数据进一步通过管道传输到下一个辅助节点。这种结构减少了主设备发送数据的开销，并以更平衡的方式利用网络带宽。In such an implementation, the first secondary in the chain will have to forward the data to the next secondary, incurring more overhead than that of the next secondary.差异并不显著。报告的数字是针对第一中学的。

对于所有三种方案，磁盘和 CPU 都接近饱和。Logical-V 的吞吐量稍高，这会导致 CPU 利用率、网络吞吐量和磁盘吞吐量更高。分层似乎稍微受到影响，因为

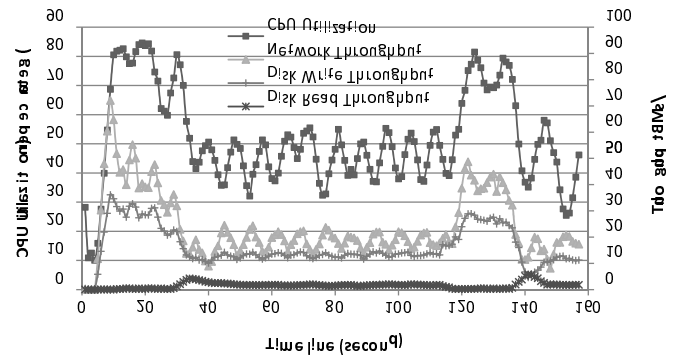


图 3：检查点和合并、逻辑复制的影响。

它必须将日志条目传递到较低层进行传输，从而中断请求处理管道。客户端观察到的吞吐量低于副本上的磁盘吞吐量，因为每条记录不仅写入日志，而且还压缩并写入检查点。Logical-V 和分层复制在辅助节点上的 CPU 利用率较低，因为辅助节点不维护内存结构或执行检查点。这两种方案的网络吞吐量较高，因为它们需要将检查点发送到辅助节点。

时间视图。在第二个实验中，我们研究了基于日志的系统由于检查点和合并而固有的时间波动。图 3 显示了逻辑复制随时间变化的 CPU 利用率、网络吞吐量和磁盘读/写吞吐量（全部在主服务器上）。

磁盘读取吞吐量的两次提升（第 30 秒和第 135 秒）标志着读取检查点和磁盘映像的合并操作的开始。合并还引入了磁盘写入与日志写入的竞争。结果是日志写入和客户端请求处理速度慢得多。这意味着较低的 CPU 利用率（由于要处理的请求较少）、较低的网络吞吐量和较低的磁盘写入吞吐量（即使有额外的写入请求源）。第一次合并在第 115 秒结束，此时系统能够处理更多客户端请求，这反映在 CPU 利用率、网络吞吐量、磁盘写入吞吐量的大幅峰值上，所有这些都是由于客户端请求处理所致。

检查点定期发生，从第 10 秒开始，甚至在合并操作期间也是如此。执行检查点时，CPU 利用率会因压缩而上升。检查点也会写入磁盘，与日志写入竞争。这会减慢日志写入和客户端请求处理速度。结果是磁盘吞吐量和网络吞吐量降低。因此，每个小波动都对应于一个检查点操作。

逻辑-V 和分层复制的图相似，因此被省略。唯一明显的区别是，这两种方案在检查点期间的网络吞吐量更高，因为检查点被发送到

Replication Scheme	Total Throughput (MB/s)	Primary Replica			Secondary Replica		
		CPU (%)	Network (MB/s)	Disk (MB/s)	CPU (%)	Network (MB/s)	Disk (MB/s)
Logical	20.3	81.6	41.5	27.5	75.2	41.0	27.2
Logical-V	21.4	94.4	60.4	28.9	31.4	51.7	28.9
Layered	19.2	83.0	53.2	25.6	22.3	46.2	25.6

表1:

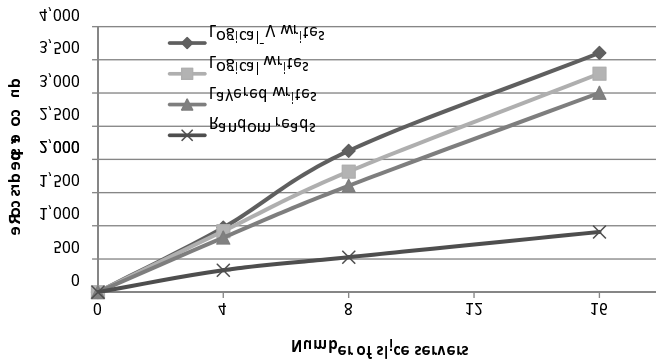


图 4: 可扩展性。

次要的。

可扩展性。在正常情况的最终实验中，我们研究了系统的可扩展性。我们将分配给每台服务器的分片数量固定为 40，并使用随机副本放置策略，为服务器提供均衡的分片和主/辅角色分配。然后，我们测量系统中不同数量的服务器的聚合客户端吞吐量，以每秒 (32KB) 记录数为单位。图 4 显示了所有这些方案近乎完美的可扩展性。由于逻辑复制在辅助副本上使用额外的资源，因此在使用多个副本组的端到端实验中，其总体吞吐量会变得较低。分层复制较低，因为该方案中每个服务器上有一个日志，并且观察到它对负载不平衡更敏感：过载的服务器似乎会影响分层复制中的更多服务器。

我们进一步测量随机键查询的可扩展性。每台服务器都是一个切片的主服务器，包含约 3GB 数据，组织在大约 56 个检查点中。我们关闭自己的缓存，但不关闭文件系统缓存或磁盘缓存。我们报告稳定状态下的查询吞吐量。由于查询仅由主节点处理，且数据放置相同，因此所有三种复制方案都会产生相同的结果。因此，我们仅报告一系列数字。由于数据共置，每个查询请求都在本地处理，这导致近乎完美的扩展。

4.2 故障、协调和恢复期间的性能

在本小节中，我们报告失败、协调和恢复期间的评估结果。第一个实验

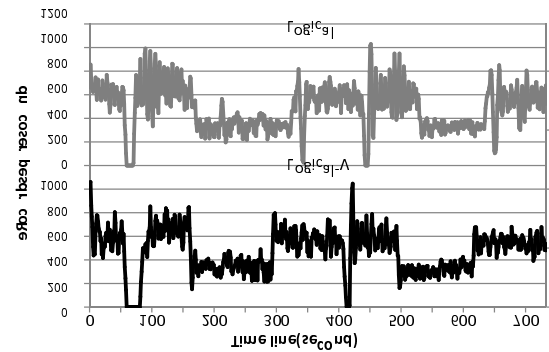


图 5: 故障、协调和恢复期间的性能。

在存储单个切片的三台服务器上使用单个副本组。图 5 显示了面对以下一系列事件时客户端感知的吞吐量（以每秒记录数表示）的变化。在第二个 60 处，主节点被杀死；服务器在第 160 秒恢复并重新加入；同一个服务器，现在是辅助服务器，在第二个 410 再次被终止；然后它恢复并在第 490 秒再次重新加入。

图 5 显示了逻辑复制和逻辑 V 复制随时间变化的结果。尽管日志处理存在差异，但分层复制的行为与逻辑-V 几乎相同，因此仅显示逻辑-V 曲线。（下一个实验区分逻辑-V 和分层复制。）

在第二个 60 处，主节点被杀死。租约将在 15 秒内到期，此时辅助节点会提出新的配置以删除有故障的主节点。逻辑复制中的协调要快得多，因为辅助副本已经维护了内存中的数据结构。这意味着更短的停机时间。对于逻辑-V 复制，新的辅助节点必须加载日志并播放日志，以便重建内存中的数据结构，然后才能接受新请求。

在第 160 秒，故障服务器恢复并重新加入。新副本的追赶恢复导致的资源消耗会导致客户端感知的吞吐量降低。当恢复在第 300 秒完成时，副本组恢复到 3 个服务器。当作为辅助服务器的服务器在第 410 秒被终止时，主服务器需要 10 秒才能重新配置以删除有故障的辅助服务器。但不需要和解。因此，我们看到的干扰较小。当服务器恢复并重新加入时，由于追赶恢复，吞吐量也会出现类似的下降。

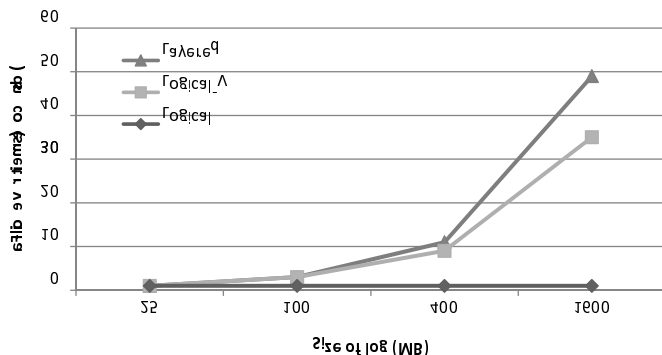


图 6：故障转移时间与日志大小

重新配置点存在波动（例如，在第 350 和 650 秒添加新的辅助节点时），特别是对于逻辑复制。这是因为当主设备联系配置管理器进行重新配置时，请求会被缓冲。当重新配置获得批准时，这些缓冲的请求将被批量处理。由于缺乏对事件发生时间的精确控制，这两个图表并不完全一致。

故障转移时间。在下一个实验中，我们将研究不同方案的故障转移时间。在分层复制中，故障转移发生在两层：在较低层，已准备好但尚未提交的请求将被重新发送。这是相对较快的，因为这些请求都在内存中。上层的故障转移涉及到读取并播放日志以重建内存中的数据结构。后者成本更高，并且取决于需要重播的日志的大小。

在Logical-V中，故障转移还涉及新主服务器上的日志处理，但处理逻辑与分层复制的处理逻辑不同。当每台服务器上有多片切片时，差异更加明显。我们使用三服务器三片配置来根据日志大小来量化故障转移时间，其中每台服务器都是一个片的主服务器，另外两台服务器作为辅助服务器。然后我们杀死一台服务器；然后，另一台服务器成为最初分配给故障服务器的分片的主服务器。我们测量不同日志大小的故障转移时间。

图 6 显示了三种方案的结果。逻辑复制性能最佳，因为新的主数据库无需重播日志即可重建内存中的数据结构。Logical-V在本地重放日志：其本地日志大约是由于日志合并而恢复的分片日志的三倍长。对于分层复制，指定的服务器加载日志条目并将它们发送到新的主服务器。这会导致远程磁盘读取，这通常是不可避免的：发生故障的服务器通常是多个分片的主服务器；指定的服务器需要处理日志并将相关日志条目发送到不同的新主服务器。但由于日志中只包含待恢复分片的日志条目，因此分层复制读取的总字节数仅为逻辑-V 的1/3。

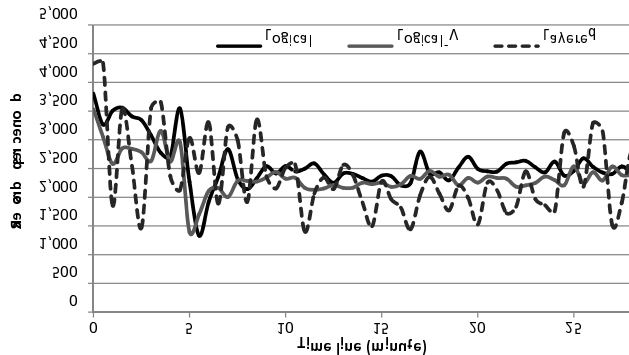


图 7：故障和恢复的全球影响。

然而，由于远程读取，分层在这个实验中仍然较差。

逻辑-V 和分层复制之间的比较并不像结果所示的那么简单。对于逻辑-V，如果每台服务器都是 n 分片的主服务器，则每台服务器上的日志平均包含 $3n$ 分片的条目。新的主节点必须读取整个日志才能获取一小部分有用的日志条目。我们维护一个内存映射来定位每个切片的日志条目，从而避免读取整个日志。当服务器发生故障时，最多可能有 n 个服务器重播其本地日志。对于分层复制，仅解析单个日志并将相关日志条目发送到适当的新主节点。总体而言，它从磁盘读取的数据较少，但可能会导致较高的延迟，因为一台服务器必须处理整个日志。

失败和恢复的全球影响。为了量化实际环境中故障的影响，我们还研究了 16 台机器集群中两台服务器被终止时的系统性能。图 7 显示了吞吐量（以每秒记录数表示）随时间的变化。在这个实验中，两台服务器在五分钟左右被杀死。对于分层复制，协调持续 5 分钟，并在第 10 分钟结束。我们等到所有协调完成后，才指示丢失副本的副本组添加随机服务器以恢复冗余级别。此过程会导致吞吐量更低，并在第 24 分钟结束。然后系统返回到正常状态，吞吐量略低于故障前的吞吐量，因为只剩下 14 台服务器。逻辑复制几乎立即完成协调，并在第 8 分钟左右完成恢复：恢复速度更快，因为此时插入系统的数据较少。逻辑-V 的曲线与逻辑复制的曲线类似。这有点令人惊讶。用于在合并日志上定位切片的日志条目的内存中映射显着减少了新主节点在逻辑 V 中读取的数据量。此外，全局协调在所有服务器上并行执行并迅速完成。这与分层复制的情况相反，分层复制需要解析两个 2 GB 日志。

摘要：总的来说，我们已经证明了复制

框架确实产生了实用的复制方案，在正常操作期间以及故障和恢复期间提供良好的性能，以及良好的可扩展性。在我们评估的三种方案中，逻辑复制在故障转移期间明显更胜一筹。这是以额外的 CPU 和内存资源为代价的，这会导致正常情况下性能较差，尤其是在充分利用 CPU 和内存的工作负载中。分层复制由于其对日志的特殊处理以及故障转移期间处理日志的额外机制而更加复杂。复杂性似乎也会转化为稍差的性能。Logical-V 更简单并且提供合理的整体性能。

5. 相关工作

复制协议已经以共识协议和复制状态机方法的形式得到了广泛的研究。代表性协议包括 Paxos [18] 和视图标记复制 [22]。

为了实现高可靠性和可用性而实现复制的分布式系统也有着悠久的历史。很难一一列举。相反，我们专注于与我们最接近的解决方案的一小部分。

HARP [20] 是最早在商用硬件上构建高可用性文件系统的公司之一。它采用主/备方式进行复制，并带有预写日志。HARP 的日志完全保存在内存中，并在提交请求之前复制到所有辅助服务器上。HARP 不需要维护磁盘日志，因为它假设每个服务器都配备了 UPS。HARP 使用视图标记复制协议 [22]。每个副本组由 $2n + 1$ 个服务器组成，其中 $n + 1$ 个服务器存储数据。其他 n 服务器是见证者，仅在重新配置（即视图更改）期间参与。

与 HARP 一样，Echo [26] 也在见证人的帮助下使用主/备份。Echo 着眼于复制的不同语义级别，尽管所有选择都位于文件系统级别及更低级别。日志用于日志记录，解决方案类似于 PacificA 中的逻辑 V 复制，尽管 Echo 不考虑日志合并。Echo 的设计考虑了协调和恢复。Echo 还对客户端和副本使用租用机制。

Petal [19] 实现了高可用性的分布式虚拟磁盘。Petal 是第一个使用 Paxos 来实现全局状态管理器的人。数据复制采用镜像。每个副本组的配置都保存在全局状态管理器中。为了协调和恢复，该方案使用繁忙位来跟踪正在进行的操作，并使用陈旧位来跟踪仅由一个副本更新的数据区域。Boxwood [21] 使用类似的技术来实现低级复制逻辑设备。与在所有服务器上实现全局状态管理器的 Petal 不同，Boxwood 使用一小组服务器。两个系统均仅支持镜像。

Frangipani [28] 是一个分层在 Petal 之上的分布式文件系统。Frangipani 使用元数据记录来清除失败 -

ure 恢复并获得更好的性能。日志在 Petal 中维护，就像我们的分层复制一样，尽管没有考虑日志合并。分布式锁服务用于协调对 Petal 存储的访问。分布式锁服务本身使用实现 Paxos 的全局状态管理器。Boxwood 中使用了类似的基于锁服务器的设计，将高级 B-Link 树抽象分层在低层可靠存储之上。在这两种情况下，不同层都不会协调共置。工作负载预计会表现出局部性，几乎没有冲突的请求，以使缓存有效。

谷歌文件系统 (GFS) [10] 是一个分布式文件系统，专为谷歌的工作负载而设计，其中文件很大并且主要是仅附加的。GFS 使用基于日志的方法来维护主节点上的元数据；主服务器以主/备份方案的变体进行复制。对于数据复制（在块服务器上），GFS 采用宽松的一致性模型，当发生故障时，副本可能具有不一致的状态。宽松的一致性模型允许复制协议不显式地处理重新配置和协调。这种选择简化了复制协议，但代价是增加了客户端处理不一致的复杂性。

Bigtable [7] 是一个分层在 GFS 之上的分布式结构化存储。PacificA 的设计受到 Bigtable 的显著影响，但也存在显著差异。Bigtable 的持久状态存储在 GFS 中；这包括日志和 SSTable（对应于图 2 中的检查点和磁盘映像）。在上层，Bigtable 使用 Chubby 锁服务 [5] 来跟踪管理器（或 tablet 服务器），其中管理器必须获得排他锁才能对 GFS 上的相关状态进行操作。Bigtable 的平板电脑主机与其平板电脑数据之间的共置只能通过尽最大努力来完成。然而，考虑到 Bigtable 构建时 GFS 已经可用，分层设计是一个明智的工程选择。

链式复制 [29] 将副本组中的副本组织成链。更新被发送到链的头部并沿着链进行处理，直到在尾部提交。尾部响应查询和更新。链式复制还使用 Paxos 来实现配置管理器。链式复制可以看作是复制框架的一种变体。将尾部视为我们框架中的主要部分，链复制提供了另一种实现重新配置不变性和提交不变性的方法。链式复制假设没有网络分区，但即使在网络分区期间也可以使用相同的租用机制来实现主不变性。每个副本可以简单地维护链上其前一个副本的租约。

以增加延迟为代价，链复制通过在头部和尾部之间分割查询和更新负载而受益。然而，当服务器参与具有不同角色的多个副本组时，好处就会减弱。对于分层复制，因为所有者服务器执行

在较低层使用链复制来维护内存中数据结构和执行检查点/合并的额外工作无助于减轻所有者服务器的开销。尽管我们的实验结果和链复制的模拟结果之间进行直接和公平的比较很困难，但根据我们的经验，我们相信链复制提供的吞吐量与我们的主/备份对应物相当，甚至更差。

与之前使用主/备份的提案不同，联合砖块阵列（FAB）[9] 举例说明了另一种基于仲裁的方法。FAB 使用新的基于多数投票的方案实现了可靠且可扩展的分布式磁盘阵列，该方案允许重新配置。重新配置是通过类似于 Paxos 的动态投票方案完成的，并且需要状态同步。

许多点对点存储系统，例如 CFS [8] 和 PAST [25]，也采用复制来提高可靠性。大多数此类系统都存储不可变的数据，这极大地简化了复制逻辑。Oceanstore [17] 和 FARSITE [1] 等系统确实允许数据突变。由于在不可信的 P2P 环境中可能存在恶意攻击，两个系统都使用拜占庭容错协议（例如[6]）。

六、结束语

复制方案是分布式系统的重要构建块，在这些系统的性能和可靠性中发挥着重要作用。本文在通用复制框架的背景下提供了一种简单明了的思考复制及其正确性的方法。同时，我们展示了如何跨越从可证明正确的方案到实际的复制方案的桥梁。我们的经验为基于日志的存储系统的复制以及复制处理故障的方式如何影响系统行为带来了新的见解。

7. 参考文献

- [1] A. Adya, W. J. Bolosky, M. Castro, G. Cermak, R. Chaiken, J. R. Douceur, J. Howell, J. R. Lorch, M. Theimer 和 R. P. Wattenhofer. FARSITE: 适用于不完全可信环境的联合、可用且可靠的存储。 *Proc. of OSDI 2002*, 第 1-14 页, 2002 年 12 月。[2] P. Alsberg 和 J. Day. 分布式资源弹性共享的原则。 *Proc. of the 2nd International Conference on Software Engineering*, 第 627-644 页, 1976 年 10 月。[3] M. Baker 和 J. K. Ousterhout. Sprite 分布式文件系统上的可用性。 *Operating System Review*, 25(2): 95-98, 1991 年 4 月。[4] N. Belaramani, M. Dahlin, L. 高, A. Nayar, A. Venkataramani, P. Yalagandula 和 J. Cheng. PRACTI 复制。 *Proc. of 3rd NSDI*, 2006 年 5 月。[5] M. Burrows. 用于松散耦合分布式系统的胖乎乎的锁服务。 *Proc. of the Seventh Symposium on Operating System Design and Implementation (OSDI 2006)*, 2006 年 1 月。[6] M. Castro 和 B. Liskov. 实用的拜占庭容错。 *Proc. of the 3rd OSDI*, 1999 年 2 月。[7] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes 和 R. E. Gruber. Bigtable: 结构化数据的分布式存储系统。在 *Proc. of the 7th OSDI*, 2006 年 11 月。

- [8] F. Dabek, M. F. Kaashoek, D. Karger, R. Morris 和 I. Stoica. 与 CFS 的广域协同存储。 *Proc. of the 18th SOSP*, 第 202-215 页, 2001 年。[9] S. Frølund, A. Merchant, Y. Saito, S. Spence 和 A. Veith. FAB: 以低廉的价格构建可靠的企业存储系统。 *Proc. 11th ASPLOS*, 2004 年 10 月。[10] S. Ghemawat, H. Gobioff 和 S. T. Leung. 谷歌文件系统。 *Proc. of the 19th SOSP*, 2003 年 10 月。[11] C. Gray 和 D. Cheriton. 租约: 一种有效的分布式文件缓存一致性容错机制。 *Proc. 12th Symp. Operating Systems Principles (SOSP)*, 第 202-210 页, 1989 年 12 月。[12] J. Gray. 有关数据库操作系统的注释。 *Operating Systems: An Advanced Course, Lecture Notes in Computer Science*, 60: 393-481, 1978。[13] R. 哈格曼. 使用日志记录和组提交重新实现 Cedar 文件系统。 *Proc. of 11th SOSP*, 第 155-162 页, 1987 年。[14] M. P. Herlihy 和 J. M. Wing. 线性化: 并发对象的正确性条件。 *ACM Trans. Program. Lang. Syst.*, 12(3):463-492, 1990。[15] M. Kazar, B. Leverett, O. Anderson, V. Apostolides, B. Buttos, S. Chutani, C. Everhart, W. Mason, S. Tu 和 E. Zayas. Decorum 文件系统架构概述。 *Proc. of the USENIX Summer'90 Conference*, 第 151-163 页, 1990 年。[16] J. J. Kistler 和 M. Satyanarayanan. Coda 文件系统断开的操作。 *Proc. of the 13th SOSP*, 第 213-225 页, 1991 年。[17] J. Kubiawicz, D. Bindel, Y. Chen, S. Czerwinski, P. Eaton, D. Geels, R. Gummadi, S. Rhea, H. Weatherspoon, W. Weimer, C. Wells 和 B. Zhu. OceanStore: 全球规模持久存储的架构。 *Proc. of 9th ASPLOS*, 第 190-201 页, 2000 年 11 月。[18] L. Lamport. 兼职议会。 *ACM Trans. on Computer Systems*, 16(2):133-169, 1998 年 5 月。[19] E. K. Lee 和 C. A. Thekkath. Petal: 分布式虚拟磁盘。 *Proc. 7th ASPLOS*, 第 84-92 页, 1996 年 10 月。[20] B. Liskov, S. Ghemawat, R. Gruber, P. Johnson, L. Shriram 和 M. Williams. Harp 文件系统上的复制。 *Proc. of 13th SOSP*, 第 226-238 页, 1991 年。[21] J. MacCormick, N. Murphy, M. Najork, C. A. Thekkath 和 L. Zhou. Boxwood: 抽象作为存储基础设施的基础。 *Proc. of the 6th OSDI*, 第 105-120 页, 2004 年 12 月。[22] B. M. Oki 和 B. H. Liskov. Viewstamped 复制: 一种支持高可用性分布式系统的新主复制方法。见 *Proc. of 7th PODC*, 第 8-17 页, 1988 年。[23] C. H. Papadimitriou. 并发数据库更新的可串行性。 *Journal of the ACM*, 26(4): 631-653, 1979 年 10 月。[24] M. Rosenblum 和 J. K. Ousterhout. 日志结构文件系统的设计与实现。 *ACM Trans. on Computer Systems*, 10(1):26-52, 1992。[25] A. Rowstron 和 P. Druschel. PAST 中的存储管理和缓存, 一种大规模、持久的点对点存储实用程序。见 *Proc. of the 18th SOSP*, 第 188-201 页, 2001 年。[26] G. Swart, A. Birrell, A. Hisgen 和 T. Mann. Echo 文件系统上的可用性。研究报告 112, 系统研究中心, 数字设备公司, 1993 年 9 月。[27] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer 和 C. H. Hauser. 管理 Bayou (弱连接的复制存储系统) 中的更新冲突。 *Proc. of the 15th SOSP*, 第 172-183 页, 1995 年 12 月。[28] C. A. Thekkath, T. P. Mann 和 E. K. Lee. Frangipani: 可扩展的分布式文件系统。 *Proc. 16th Symp. Operating Systems Principles (SOSP)*, 第 224-237 页, 1997 年 10 月。[29] R. van Renesse 和 F. B. Schneider. 链式复制支持高吞吐量和可用性。见 *Proc. of the 6th OSDI*, 第 91-104 页, 2004 年。[30] H. Yu 和 A. Vahdat. 设计和评估基于 conit 的复制服务持续一致性模型。 *ACM Trans. Comput. Syst.*, 20(3): 239-182, 2002。